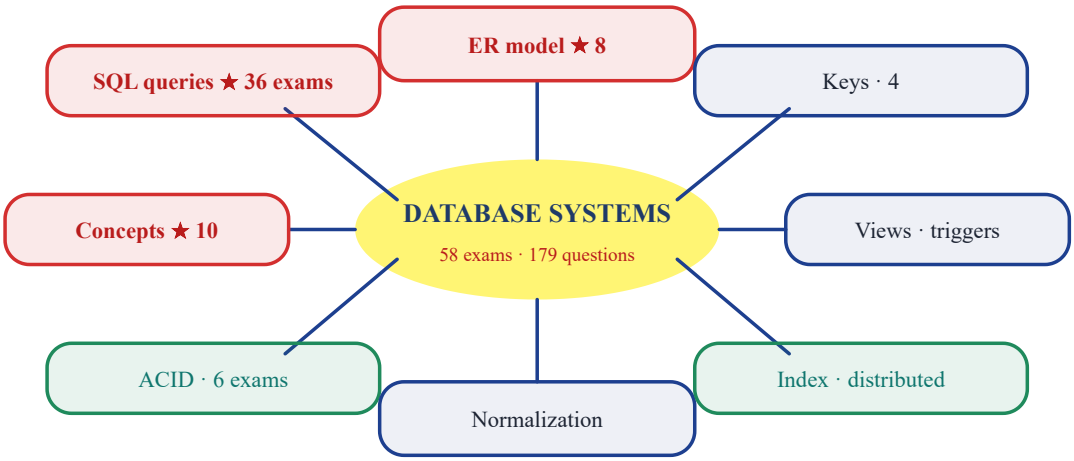
Database Systems

every DBMS question asked in BCS · NTRCA · BPSC · bank · BUET-set papers, solved in 16 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
>
> ✎ **Asked in:** the exams it was taken from, e.g. BPSC AP 2023 · Milk Vita AM(CSE) 2023

No dated exam → the card names its source instead (NTRCA question bank, coaching suggestion sheet, bank IT viva). Then: answer → green = output on the sample rows → trap.

Most asked

① schema → SQL query
② GROUP BY + HAVING
③ subquery vs average
④ self-join (manager)
⑤ ACID (5 exams)
⑥ ER → tables (M:N)

Syllabus coverage

"**Concepts of database systems**; Data Models: **Entity-Relationship model**, Relational model; Query Languages: Relational algebra, **SQL**; **Constraints and triggers**; Functional dependencies and **normalization**; Indexing: primary and secondary indexes, B+ trees, hash tables; **Transaction management**; Recovery; Concurrency control; Access control and security".

> **Legend:** **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = output / worked answer

Contents

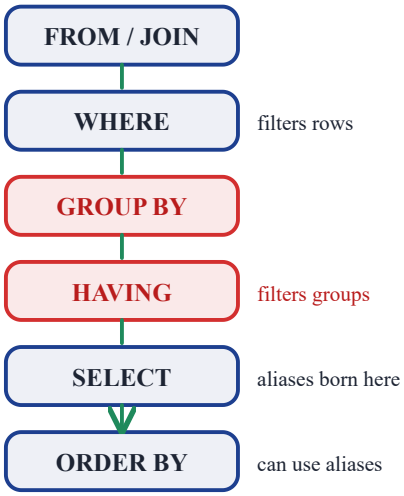
✓ SQL I: GROUP BY, HAVING …… p. 2–3
✓ SQL II: subqueries, Nth salary … p. 3–4
✓ SQL III: joins, self-joins …… p. 4–5
✓ ER model II: worked ERDs, keys … p. 10
✓ DBMS concepts, ACID, recovery … p. 11
✓ Normalization, index, NoSQL … p. 12

✓ SQL IV: DDL/DML, outputs …… p. 6–7
✓ Views, triggers, sub-languages … p. 8
✓ ER model I: notation, mapping … p. 9
✓ Night-before sheet …… p. 12
✓ Book drills: salary, self-join, ER … p. 13–15
✓ Practice MCQs + written …… p. 16

1 GROUP BY + HAVING: the most-asked pattern

Sample rows used on every SQL sheet (`employees ⋈ department`):

name	job	salary	dept	mgr
Karim	Manager	90000	IT	–
Rahim	Programmer	60000	IT	Karim
Sumi	Programmer	58000	IT	Karim
Nila	Analyst	60000	IT	Karim
Babu	Manager	70000	HR	–
Tania	Officer	40000	HR	Babu
Jamal	Officer	38000	HR	Babu
Ruma	Accountant	45000	Accounts	Babu
Hasan	Accountant	75000	Accounts	Babu



logical execution order

Company average = 536000 / 9 ≈ **59556**

Q1: Analyze the following code: what does it return?

✎ **Asked in:** Combined Bank Officer(IT) 2023 · Rupali Bank ANE 2023 · also on the Petrobangla ANE suggestion sheet

```
SELECT d.department_name,
       AVG(e.salary) AS average_salary
FROM employees e
JOIN department d
  ON e.department_id = d.department_id
WHERE e.salary >
      (SELECT AVG(salary) FROM employees)
GROUP BY d.department_name
HAVING COUNT(*) > 2
ORDER BY average_salary DESC;
```

1. WHERE keeps employees paid **above the company average** (59556): Karim, Rahim, Nila, Babu, Hasan
2. GROUP BY department: IT 3, HR 1, Accounts 1
3. HAVING keeps groups with **more than 2** such employees
4. Shows their average, highest first

→ IT | 70000 (only IT has 3 above-average employees: 90000 + 60000 + 60000)

Exam trap: As printed, the query is **invalid**: alias `e` is used but never declared (`FROM employees` needs `e`), and the suggestion-sheet copy has an empty `COUNT()`. Say so, fix it, then explain. The suggestion-sheet version compares with the **own department's** average (correlated subquery): on these rows it returns **no row**.

Memory trick: "**WHERE before, HAVING after**": WHERE cannot use AVG/COUNT, HAVING can. An alias from SELECT works only in ORDER BY.

Same pattern, other schemas: courses with **exactly 2** / **more than 2** offerings → `GROUP BY c.crscode HAVING COUNT(*) = 2` ✎ **Asked in:** Petrobangla AM(CSE) 2024 · courses and student count where enrolment ≤ 30 → `HAVING COUNT(*) <= 30` ✎ **Asked in:** ICB Securities AP 2026

Q2: COURSE(Course_ID, Course_name, Chief_ID), INSTRUCTOR(Instructor_ID, Course_ID, Salary, Joining_date): average salary for each course.
✎ **Asked in:** NACTAR 2020 · same with departments: NESCO JAM(ICT) 2021

```
SELECT c.Course_name, AVG(i.Salary) AS avg_salary
FROM COURSE c JOIN INSTRUCTOR i ON i.Course_ID = c.Course_ID
GROUP BY c.Course_ID, c.Course_name;
```

Q3: Instructor(ID, name, dept_name, salary): (a) average salary per department; (b) departments whose average is above 42000; (c) instructors earning more than **some** (at least one) CSE instructor.
✎ **Asked in:** NRCC AP 2021

```
(a) SELECT dept_name, AVG(salary)
    FROM instructor GROUP BY dept_name;
(b) … GROUP BY dept_name
    HAVING AVG(salary) > 42000;
(c) SELECT name FROM instructor
    WHERE salary > SOME
     (SELECT salary FROM instructor
      WHERE dept_name = 'CSE');
```

Rows: CSE 50000, 40000 · EEE 45000, 38000 · Physics 41000, 47000
(a) CSE 45000, EEE 41500, Physics 44000
(b) CSE, Physics
(c) everyone above **40000**: Arif, Chand, Emon, Faria

Trick: **> SOME** = > MIN · **> ALL** = > MAX

2 Subqueries: compare with an average or a person

Q4: EMPLOYEE(id, name, department, salary): find employees who work in a department whose average salary is lower than the average salary of all departments.
✎ **Asked in:** BPDB AE(CSE) 2023

```
SELECT name, department_id, salary FROM employees
WHERE department_id IN
  (SELECT department_id FROM employees GROUP BY department_id
   HAVING AVG(salary) < (SELECT AVG(salary) FROM employees));
```

→ Babu, Tania, Jamal (HR average 49333 < 59556)

Q5: (a) Employees with the **same salary as Rahim** but not the same job. (b) Employees paid more than the company average.

✎ **Asked in:** BGDCL AM(CSE) 2024

```
(a) SELECT name FROM employees
    WHERE salary = (SELECT salary FROM employees WHERE name = 'Rahim')
      AND job <> (SELECT job FROM employees WHERE name = 'Rahim');
(b) SELECT name FROM employees
    WHERE salary > (SELECT AVG(salary) FROM employees);
```

→ (a) Nila (60000, Analyst) · (b) Karim, Rahim, Nila, Babu, Hasan

Q6: employees(id, first_name, …, salary, dept_id), department(dept_id, dept_name): first name and department of the employee whose salary is maximum.

✎ **Asked in:** BIWTA AE(CSE) 2023

```
SELECT e.first_name, d.dept_name
FROM employees e JOIN department d ON e.dept_id = d.dept_id
WHERE e.salary = (SELECT MAX(salary) FROM employees);
```

→ Karim | IT

2nd / 3rd highest salary

```
-- 2nd highest
SELECT MAX(salary) FROM employees
WHERE salary <
  (SELECT MAX(salary) FROM employees);
-- Nth highest (N = 3)
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

→ 75000 · 70000

✎ **Asked in:** BCC AP 2017; 3rd highest: NTRCA question bank

With a window function

```
SELECT name, salary FROM
 (SELECT name, salary,
   DENSE_RANK() OVER
     (ORDER BY salary DESC) AS rnk
  FROM employees) x
WHERE rnk = 3;
```

Trap: Without **DISTINCT** / DENSE_RANK, two people on 60000 make "3rd highest" wrong. Oracle uses `FETCH FIRST`, SQL Server `TOP`.

3 Joins and self-joins

Q7: Employees who report to the manager whose first name is 'abc': return first name, last name, employee ID and salary.

✎ **Asked in:** PGCL SAE 2021

```
SELECT e.first_name, e.last_name, e.id, e.salary
FROM employees e JOIN employees m ON e.manager_id = m.id
WHERE m.first_name = 'abc';
```

→ with 'Babu' as the manager: Tania, Jamal, Ruma, Hasan

Q8: Employees whose salary is greater than their manager's.

✎ **Asked in:** NESCO AM(ICT) 2021

```
… JOIN employees m
  ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

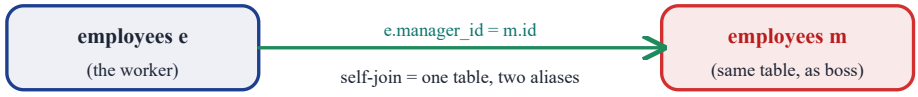
→ Hasan (75000 > Babu 70000)

Q9: Employees whose manager is in the same region.

✎ **Asked in:** DPDC AM(ICT) 2025

```
… JOIN employees m
  ON e.manager_id = m.id
WHERE e.region = m.region;
```

→ Rahim, Nila, Tania, Hasan



Exam trap (Q4–Q5): `WHERE salary > AVG(salary)` is an **error**: an aggregate needs its own subquery. If the subquery can return several rows (two Rahims), `=` fails: use `IN`.

Q10: Project(Project_id, Project_name, Manager_name), Location(location_id, Location_name, project_id), Employee(Employee_id, Employee_Name, Location_id, Joining_date, Salary): show project_name, Location_name and total salary of each project's employees who joined before January 2021.

✎ **Asked in:** APSCL AE(ICT) 2021

```
SELECT p.Project_name, l.Location_name, SUM(e.Salary) AS Total_salary
FROM Project p
JOIN Location l ON l.project_id  = p.Project_id
JOIN Employee e ON e.Location_id = l.location_id
WHERE e.Joining_date < '2021-01-01'
GROUP BY p.Project_name, l.Location_name;
```

→ Payroll | Dhaka | 50000 · e-Banking | Ctg | 60000 · e-Banking | Dhaka | 45000

Q11: employee(id, name, salary), equipment(id, name, price), hire(employee_id, equipment_id): name of the employee who borrowed the most equipment.

✎ **Asked in:** BAPEX AGM(ICT) 2023

```
SELECT e.name, COUNT(*) AS items
FROM employee e JOIN hire h ON h.employee_id = e.id
GROUP BY e.id, e.name
HAVING COUNT(*) = (SELECT MAX(cnt) FROM
        (SELECT COUNT(*) AS cnt FROM hire GROUP BY employee_id) t);
```

→ Chand | 3 (ties are all shown: safer than ORDER BY … LIMIT 1)

Q12: Product(pname, price, category, maker), Purchase(buyer, seller, stock), Company(cname, country), Person(per-name, address, phone); maker = cname, buyer/seller = per-name. (a) Persons who bought a product made by an American company. (b) Sum of all products and of all sold products by country.

✎ **Asked in:** NWPGCL AM(ICT) 2024

```
(a) SELECT DISTINCT pu.buyer FROM Purchase pu
    JOIN Product p ON p.pname = pu.stock
    JOIN Company c ON c.cname = p.maker
    WHERE c.country = 'USA';
(b) SELECT c.country,
      (SELECT SUM(p2.price) FROM Product p2 JOIN Company c2
         ON c2.cname = p2.maker WHERE c2.country = c.country) AS all_products,
      SUM(p.price) AS sold_value
    FROM Purchase pu JOIN Product p ON p.pname = pu.stock
    JOIN Company c ON c.cname = p.maker GROUP BY c.country;
```

→ sample: Apple iPhone 1000 + Dell XPS 1500 (USA), Walton Primo 200 (Bangladesh), Primo sold twice: (a) Rafi, Shoeb · (b) Bangladesh | 200 | 400 · USA | 2500 | 2500

Assumption to write: "American category" is read as **made by a company whose country is USA**; Purchase.stock is read as the product name. State such readings in one line: examiners reward it.

Q13: Actor(actor_id, name, age), Play(play_id, writer), Role(play_id, actor_id): (i) number of actors who acted in **3 or more** plays by Shakespeare; (ii) actors in a Chekhov play but never in a Shakespeare play.

✎ **Asked in:** Petrobangla ANE suggestion sheet (no dated paper)

```
(i)  SELECT COUNT(*) FROM (SELECT r.actor_id FROM Role r
       JOIN Play p ON p.play_id = r.play_id WHERE p.writer = 'Shakespeare'
       GROUP BY r.actor_id HAVING COUNT(DISTINCT r.play_id) >= 3) t;
(ii) SELECT name FROM Actor
     WHERE actor_id IN     (… WHERE p.writer = 'Chekhov')
       AND actor_id NOT IN (… WHERE p.writer = 'Shakespeare');
```

Memory trick: "**in A but never B**" = IN … AND NOT IN … (or EXCEPT). "**at least n**" = GROUP BY + HAVING COUNT ≥ n.

4 Create, insert, update, delete

Q14: Employee(ID, First_name, Last_name, Joining_date, Salary), Employee_Info(Emp_ID, Designation): (a) create both tables; (b) designation-wise min, max, average salary; (c) employees who joined before February 2023.

✎ **Asked in:** MRA SCO 2023

```
CREATE TABLE Employee (ID INT PRIMARY KEY, First_name VARCHAR(30),
  Last_name VARCHAR(30), Joining_date DATE, Salary DECIMAL(10,2));
CREATE TABLE Employee_Info (Emp_ID INT REFERENCES Employee(ID),
  Designation VARCHAR(30));
SELECT i.Designation, MIN(e.Salary), MAX(e.Salary), AVG(e.Salary)
FROM Employee e JOIN Employee_Info i ON i.Emp_ID = e.ID GROUP BY i.Designation;
SELECT * FROM Employee WHERE Joining_date < '2023-02-01';
```

The paper says "department-wise", but neither table has a department column: group by Designation and say so.

Insert one row ✎ **Asked in:** NESCO AM(ICT) 2018; BPSC AP 2021

```
INSERT INTO Employee
VALUES (1,'Rafi','Ahmed',
 '2023-01-10', 45000);
```

+10 % raise for HR ✎ **Asked in:** PBL SO 2018

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id IN (SELECT
 department_id FROM department
 WHERE department_name='HR');
```

Show only the duplicates ✎ **Asked in:** ABL Officer(ICT) 2017 · BBA AME 2025

```
SELECT name, gpa, COUNT(*)
FROM student GROUP BY name, gpa
HAVING COUNT(*) > 1;
```

Remove duplicates, keep one ✎ **Asked in:** NU AP 2020

```
DELETE FROM student WHERE id
NOT IN (SELECT MIN(id) FROM student
 GROUP BY name, gpa);
```

First 3 letters of a name ✎ **Asked in:** DPDC JAM(ICT) 2020 · BCPSC AP 2018: `SELECT SUBSTRING(first_name, 1, 3) FROM worker;` (also `LEFT(first_name, 3)`; Oracle `SUBSTR`) · **Top 10 % rows** ✎ **Asked in:** CB AP 2018 · Combined Bank AME 2019 (per department): `SELECT TOP 10 PERCENT …` (SQL Server) or `NTILE(10) OVER (PARTITION BY dept ORDER BY gpa DESC) = 1`.

5 Predict-the-output questions

Q15: t(val) holds 1, 2, 3, NULL, NULL, 4, 5. Output of `COUNT(*)` and `COUNT(DISTINCT val)`?

✎ **Asked in:** BCC AP 2021

→ **7** and **5**: COUNT(*) counts rows, COUNT(col) skips NULL

Q16: Loan_Records: (Rahim, Shuvo, 1000), (Shamim, Rasel, 500), (Mamun, Shuvo, 7000). Rows in S(Borrower, Bank_Manager) NATURAL JOIN T(Bank_Manager, Loan_Amount)?

✎ **Asked in:** BAUST AP 2021

→ **5**: Shuvo 2 × 2 = 4, Rasel 1 × 1 = 1

Exam trap: NATURAL JOIN matches on **every** column with the same name, not on a key, so repeated values multiply rows. The printed query has a stray comma (`Bank_Manager, FROM`): note it, then answer.

Q17: Movies(mid, title, year), People(pid, name), Genres(gid, genre), HasRole(pid, mid, role), HasGenre(gid, mid): number of movies that are romantic comedies.

✎ **Asked in:** BB AP 2023

```
SELECT COUNT(*) FROM
 (SELECT hg.mid FROM HasGenre hg JOIN Genres g ON g.gid = hg.gid
  WHERE g.genre IN ('Romance', 'Comedy')
  GROUP BY hg.mid HAVING COUNT(DISTINCT g.genre) = 2) t;
```

Exam trap: `WHERE genre = 'Romance' AND genre = 'Comedy'` returns **0**: one row can't hold two values. Ask for **both** with GROUP BY + HAVING COUNT(DISTINCT …) = 2.

Q18: Movies(title, year, length, genre, studioName, producerCert), Studio(name, address): (a) address of the studio MGM; (b) number of movies not produced by MGM.

✎ **Asked in:** BPSC AME 2021

```
(a) SELECT address FROM Studio WHERE name = 'MGM';
(b) SELECT COUNT(*) FROM Movies WHERE studioName <> 'MGM' OR studioName IS NULL;
```

6 Views and triggers

Q19: Create a view showing the employees whose salary equals the maximum, minimum or average salary.

✎ **Asked in:** Milk Vita AM(CSE) 2023

```
CREATE VIEW salary_marks AS
SELECT * FROM employees
WHERE salary IN ((SELECT MAX(salary) FROM employees),
```

```
                (SELECT MIN(salary) FROM employees),
                (SELECT AVG(salary) FROM employees));
```

→ Karim (90000), Jamal (38000); nobody earns exactly the average 59556

> **Q20:** Employee(emp_id, emp_name, dept_id, salary), Department(dept_id, dept_name, increment_dept): create a trigger that increases an employee's salary by 10 % when it is above 30000.
> ✎ **Asked in:** PGCB AE(CSE) 2021

```
CREATE TRIGGER raise_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
  IF NEW.salary > 30000 THEN
    SET NEW.salary = NEW.salary * 1.10;
  END IF;
END;            -- MySQL syntax
```

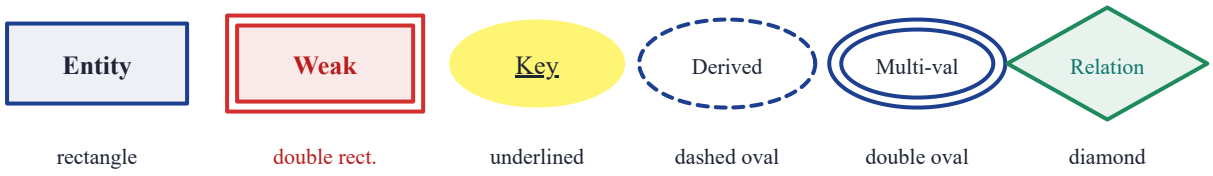
Insert 25000 → stays 25000
Insert 40000 → stored 44000

A **trigger** runs **automatically** on INSERT / UPDATE / DELETE; a **view** is a **stored SELECT** (a virtual table, no own data unless materialized).

Sub-language	Commands	Remember
DDL (Definition)	CREATE, ALTER, DROP, TRUNCATE, RENAME	changes structure, auto-commits
DML (Manipulation)	INSERT, UPDATE, DELETE (+ SELECT = DQL)	changes rows, can roll back
DCL / **TCL**	GRANT, REVOKE / COMMIT, ROLLBACK, SAVEPOINT	rights / transactions

✎ **Asked in:** full meaning of DDL and DML: BKKB AP 2017 · **DELETE** removes chosen rows (WHERE, logged, can roll back) · **TRUNCATE** removes all rows, keeps the table · **DROP** removes the table itself.

7 ER notation and mapping rules



Chen notation: double diamond = identifying relationship · double line = total participation

ER element	Becomes in tables
Strong entity	one table; key attribute → **primary key**
1 : N relationship	PK of the "1" side goes as a **foreign key** into the "N" side
M : N relationship ★	**a new table**: FKs to both sides, together the **composite PK**, plus the relationship's own attributes
Weak entity	table with owner's PK as FK; PK = owner PK + **partial key**
Multivalued attribute	separate table (entity PK, value)

Q21: While converting an E-R diagram into tables, how is a many-to-many relationship set between entities A and B converted?

✎ **Asked in:** BPSC SAE 2021

Answer: tables A(a_id, …), B(b_id, …) and a third table **R(a_id, b_id, attributes of R)**; a_id → A, b_id → B, and (a_id, b_id) together is the primary key. One extra column in A or B cannot hold "many" values.

Q22: What is a weak entity? Find a weak entity and its identifying relationship for a school database: Student(student_id, student_name, admission_year), Teacher(…), Course(course_id, subject_name, credit).

✎ **Asked in:** BCC AP 2021

A **weak entity** has **no key of its own**; it is identified only through an owner entity. Here none of the three is weak (each has its own id). Add one: **Guardian(name, relation)**, owned by Student through the identifying relationship **HAS_GUARDIAN**; its PK = (student_id, name).

Q23: Library: is Book–Borrower 1-to-1, 1-to-many or many-to-many? Justify.

✎ **Asked in:** Bank AP written (bank name cut off on the sheet)

At one time a copy has ≤ 1 borrower and a borrower many copies: **1 : N**. Over time (loan history) it is **M : N** via a LOAN table.

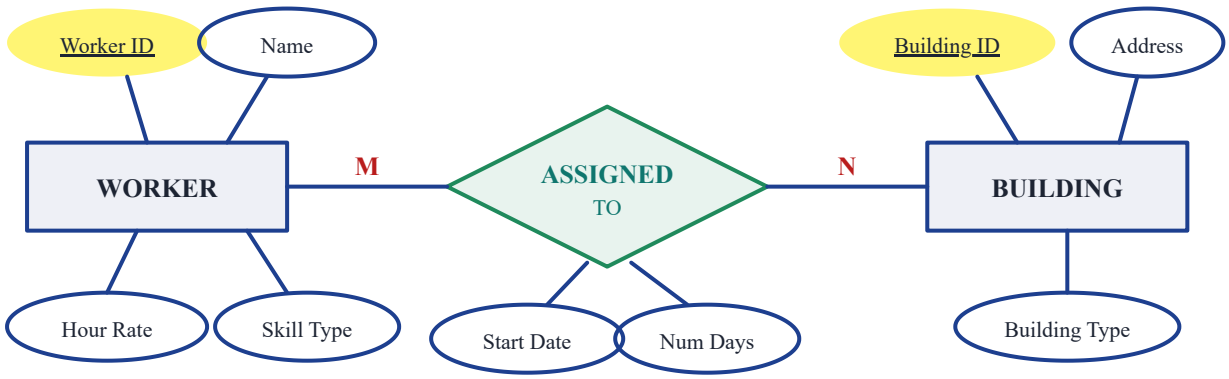
Q24: Course(c_id, c_name, total_credit), Program(prog_no, prog_name); a course is in many programs and a program has many courses.

✎ **Asked in:** BUET M.Sc.

M : N → Course, Program, **Offers(c_id, prog_no)**.

Q25: Draw the E-R diagram: Worker(Worker ID, Worker Name, Hour Rate, Skill Type), Assignment(Worker ID, Building ID, Start Date, Num Days), Building(Building ID, Address, Building Type). 5 marks

✎ **Asked in:** 18th NTRCA College ICT 2024 (Q9b)



ASSIGNMENT is the M:N relationship; Start Date and Num Days belong to the relationship, not to either entity

1. Two strong entities, WORKER and BUILDING, keys underlined 1
2. A worker works on many buildings and a building has many workers: **M : N** 2
3. Assignment's key (Worker ID, Building ID) is **composite**; each part is also a foreign key 2

Same skill, other papers: employee–equipment–**hire** (M:N, hire is the relationship table) ✎ **Asked in:** BAPEX AGM(ICT) 2023 · scenario → entities, ERD, schemas with PK and FK ✎ **Asked in:** BSCL AE(CSE) 2026 · Product/Purchase/Company/Person ERD ✎ **Asked in:** NWPGCL AM(ICT) 2024

8 Keys

Key	Meaning	In Employee(emp_id, nid, email, name, dept_id)
Super key	any attribute set that identifies a row	{emp_id}, {emp_id, name}, {nid} …
Candidate key	**minimal** super key	emp_id, nid, email
Primary key	the candidate chosen; unique, not NULL	emp_id
Alternate key	candidate keys not chosen	nid, email
Foreign key	refers to a PK of another table	dept_id → Department
Composite / surrogate	PK of 2+ columns / system-made id	(emp_id, project_id) / AUTO_INCREMENT

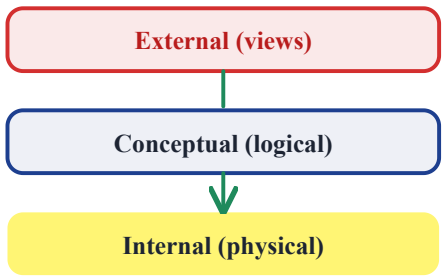
MCQ: Which key is the minimal super key? (Primary / Secondary / Foreign / Candidate)
✎ **Asked in:** 49th BCS ICT 2025
→ Candidate key

Memory trick: **Super ⊇ Candidate ⊇ Primary**: every primary key is a candidate key, every candidate key a super key, never the other way.

9 DBMS concepts and architecture

Database / DBMS: A **database** is an organised collection of related data; a **DBMS** is the software that stores, queries and protects it (MySQL, Oracle, PostgreSQL). ✎ **Asked in:** What is a database? EGCB SAE(ICT) 2023 · BKKB AP 2017 · DBMS classification: 40th BCS

File system	DBMS
redundancy, inconsistency	controlled redundancy
no concurrent safety	concurrency control
no crash recovery	backup & recovery (logs)
file-level security	users, roles, GRANT
program-data dependent	data independence



three-schema architecture

✎ **Asked in:** DBMS vs file system: 17th NTRCA ICT 2023 · external level = user view: BPDB AE(CSE) 2021 · **Degree** = no. of **columns**, cardinality = no. of rows ✎ **Asked in:** BCC AP 2023 · relationship types 1:1, 1:N, M:N ✎ **Asked in:** Combined Bank AP 2024 · DBMS-provided security (roles, views, audit, one place) vs application-provided (finer rules, repeated per app, can be bypassed) ✎ **Asked in:** BPSC AP 2022 · 38th BCS

10 Transactions and ACID

Q26: What are the ACID properties of a transaction that ensure data reliability and integrity? Explain with an example.

✎ **Asked in:** BPSC AP 2023 · Milk Vita AM(CSE) 2023 · BBL AM(ICT) 2019 · 17th NTRCA ICT 2023 · 18th NTRCA College ICT 2024

Property	Transfer 5000 from A to B
Atomicity	debit A and credit B both, or neither (ROLLBACK)
Consistency	A + B total is the same before and after
Isolation	another teller never sees A debited, B not yet credited
Durability	after COMMIT it survives a power cut (log)

Memory trick: "**All Cash Is Durable**": Atomicity by undo log · Consistency by constraints · Isolation by locks · Durability by redo log.

MCQs asked:

Cascadeless schedule prevents → reading **uncommitted** data ✎ **Asked in:** 49th BCS ICT 2025

Locking protocols are for → **concurrency control** ✎ **Asked in:** BPDB AE(CSE) 2021

WAL → log record flushed **before** the data page ✎ **Asked in:** 49th BCS ICT 2025

11 Normalization in one table

Q27: What is normalization? How many types? Explain. 5 marks

✎ **Asked in:** 18th NTRCA College ICT 2024 (Q9a) · NTRCA question bank

Form	Rule	Fix in Order(order_id, product_id, product_name, cust_id, cust_city)
1NF	atomic values, no repeating groups	one product per row, not "pen, ink"
2NF	1NF + no **partial** dependency on part of a composite key	product_name depends only on product_id → Product table
3NF	2NF + no **transitive** dependency	order → cust_id → cust_city → Customer table
BCNF	every determinant is a **candidate key**	stricter 3NF; removes the last anomalies

Purpose: remove **redundancy** and the **insert, update and delete anomalies**. Higher forms: 4NF (multivalued), 5NF (join dependency).

Indexing speeds a query by searching a sorted structure (**B+ tree**, O(log n)) instead of scanning every row; cost: extra space, slower INSERT/UPDATE. ✎ **Asked in:** BSCL AE(CSE) 2025 · Bitmap index suits

Distributed DB: one logical DB stored at several sites; advantages: local autonomy, reliability, faster local access ✎ **Asked in:** Combined Bank ADA 2022 · BCC AP 2021 · BB AME 2019 · **RDBMS vs NoSQL**: fixed

low-cardinality columns (gender, status) ✎ **Asked in:** 49th BCS ICT 2025

schema, SQL, ACID, vertical scaling vs flexible schema, key-value/document, BASE, horizontal scaling ✎ **Asked in:** Sonali Bank Officer(IT) 2021

★ Night before the exam

1. Write order: SELECT FROM WHERE GROUP BY HAVING ORDER BY
2. Run order: FROM → WHERE → GROUP → HAVING → SELECT → ORDER
3. Aggregate in WHERE ✗ → use a subquery or HAVING
4. "compare with average" → `> (SELECT AVG…)`
5. "manager / same table" → self-join e, m
6. "at least n" → HAVING COUNT ≥ n
7. "in A, never B" → IN … AND NOT IN …
8. Nth highest → DISTINCT … LIMIT 1 OFFSET N-1
9. COUNT(*) counts NULL rows, COUNT(col) doesn't
10. M:N → new table with composite PK
11. Candidate key = minimal super key
12. ACID: undo log, constraints, locks, redo log

Top traps: fix a broken printed query before explaining it · NATURAL JOIN multiplies on repeated values · `genre='A' AND genre='B'` is always empty · 2PL (grow, then shrink) ≠ deadlock-free; strict 2PL → cascadeless · DELETE vs TRUNCATE vs DROP

12 Book drills: salary queries and self-joins

These come from the Super ICT question book, which prints **no exam names**, so each card names its source instead. Every query below was run on these rows:

emp_id	name	dept_id	salary	manager_id
101	Rahim	1	90000	NULL
102	Karim	1	70000	101
103	Hasan	2	65000	101
104	Jannat	2	50000	102
105	Rafi	3	55000	102
106	Nila	3	48000	103
107	Salma	1	70000	101
108	Karim	3	68000	103

dept_id	dept_name
1	ICT
2	HR
3	Finance

Read the rows first
two people on **70000**
two people called **Karim**
Rahim has no manager

B1: From Employee(emp_id, emp_name, salary): (a) the highest salary, (b) the second-highest salary, (c) salaries that occur more than once.

✎ **Source:** Super ICT book, Q3 · 2nd highest also BCC AP 2017 (p. 4)

```
(a) SELECT name, salary FROM Employee
    WHERE salary =
      (SELECT MAX(salary) FROM Employee);
(b) SELECT MAX(salary) FROM Employee
    WHERE salary <
      (SELECT MAX(salary) FROM Employee);
```

```
(c) SELECT salary, COUNT(*) AS n
    FROM Employee GROUP BY salary
    HAVING COUNT(*) > 1;
```

→ (a) Rahim | 90000 · (b) 70000 · (c) 70000 | 2

B2: Self-join on the same table: (1) every employee with the manager's name; (2) employees whose manager earns more than they do; (3) pairs of employees with the same salary.

✎ **Source:** Super ICT book, descriptive ch. 1 · reverse of NESCO AM(ICT) 2021 (p. 4)

```
(1) SELECT e.name AS employee, m.name AS manager
    FROM Employee e LEFT JOIN Employee m ON e.manager_id = m.emp_id;
(2) … JOIN Employee m ON e.manager_id = m.emp_id WHERE m.salary > e.salary;
(3) SELECT e1.name AS a, e2.name AS b, e1.salary FROM Employee e1
    JOIN Employee e2 ON e1.salary = e2.salary AND e1.emp_id < e2.emp_id;
```

→ (1) 8 rows, Rahim | NULL first · (2) everyone but Rahim and Karim 108 (68000 > Hasan's 65000) · (3) Karim | Salma | 70000

Exam trap: (1) needs **LEFT** JOIN, or the top boss disappears. (3) needs `e1.emp_id < e2.emp_id`: with `<>` each pair prints twice, and with no condition everyone pairs with himself.

B3: The department with the highest average salary.

✎ **Source:** Super ICT book, descriptive ch. 1

```
SELECT d.dept_name, AVG(e.salary) AS avg_sal
FROM Employee e JOIN Department d ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY avg_sal DESC LIMIT 1;
```

→ ICT | 76666.67 (HR 57500, Finance 57000)

13 Ten practice problems (same rows as p. 13)

P1 3rd-highest *distinct* salary

```
WHERE salary = (SELECT DISTINCT salary
 FROM Employee ORDER BY salary DESC
 LIMIT 1 OFFSET 2)
```

→ Karim | 68000

P2 Above own department's average

```
WHERE e.salary > (SELECT AVG(x.salary)
 FROM Employee x WHERE x.dept_id = e.dept_id)
```

P6 More than *at least one* Finance employee

```
WHERE salary > ANY (SELECT salary …
  WHERE dept_name = 'Finance') -- = > MIN
```

→ everyone except Nila (48000)

P7 Departments above the company average

```
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) >
  (SELECT AVG(salary) FROM Employee)
```

→ Rahim, Hasan, Karim 108

P3 Highest *total* salary: same as B3 with `SUM`

→ ICT | 230000

P4 2nd-highest paid in every department

```
SELECT dept_id, name, salary FROM
 (SELECT dept_id, name, salary, DENSE_RANK()
   OVER (PARTITION BY dept_id
         ORDER BY salary DESC) r
  FROM Employee) t WHERE r = 2;
```

→ 1: Karim, Salma · 2: Jannat · 3: Rafi

P5 Earns more than *every* HR employee

```
WHERE salary > ALL (SELECT salary …
  WHERE dept_name = 'HR')  -- = > MAX
```

→ Rahim, Karim, Salma, Karim 108

→ ICT (company average 64500)

P8 Highest salary *without* MAX()

```
WHERE NOT EXISTS (SELECT 1
  FROM Employee x
  WHERE x.salary > e.salary)
```

→ Rahim | 90000

P9 Duplicate names

```
GROUP BY name HAVING COUNT(*) > 1
```

→ Karim | 2

P10 Top 3 salaries per department, ties kept: P4 with `WHERE r <= 3`

→ ICT: Rahim, Karim, Salma (rank 2 shared)

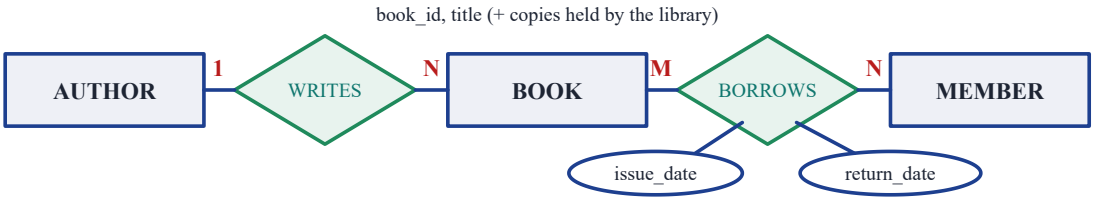
Memory trick: **> ALL = > MAX**, **> ANY = > MIN**. Ties: **DENSE_RANK** keeps them; ROW_NUMBER drops them.

Exam trap: Write the **GROUP BY** on every non-aggregated column you SELECT (`d.dept_id, d.dept_name`); MySQL may let it pass, PostgreSQL and Oracle reject it.

14 Library ER and the aptitude-test schema

B4: ER diagram for a library: a library has many books; each book has one author; members borrow many books; each loan keeps the issue and return dates.

✎ **Source:** Super ICT book, Q45 · same idea: Bank AP written (p. 9)



AUTHOR 1:N BOOK → author_id goes into Book · BOOK M:N MEMBER → a **Loan** table

Tables: Author(author_id, name) · Book(book_id, title, author_id → Author) · Member(member_id, name) · **Loan(loan_id, book_id → Book, member_id → Member, issue_date, return_date)**; loan_id lets a member borrow the same book again.

B5: Department(dept_id, dept_name), Employee(emp_id, name, dept_id, salary, joining_date): (a) insert rows, (b) employees with their department names, (c) average salary per department, (d) employees above their department's average. aptitude test

✎ **Source:** Super ICT book, aptitude Q2

```
(a) INSERT INTO Department VALUES (1,'ICT');
    INSERT INTO Employee VALUES
     (101,'Rahim',1,90000,'2018-03-01');
(b) SELECT e.name, d.dept_name FROM Employee e
    JOIN Department d ON d.dept_id = e.dept_id;
```

```
(c) SELECT d.dept_name, AVG(e.salary)
    FROM Department d JOIN Employee e
      ON e.dept_id = d.dept_id
    GROUP BY d.dept_id, d.dept_name;
(d) = P2 above
```

→ (c) ICT 76666.67 · HR 57500 · Finance 57000

Which pattern for which wording

The question says …	Write	See
"second / Nth highest"	`LIMIT 1 OFFSET N-1`	B1(b), P1
"duplicate / more than once"	`GROUP BY col HAVING COUNT(*) > 1`	B1(c), P9
"manager / same table"	self-join `e`, `m`	B2
"than **their own** dept average"	correlated subquery	P2, B5(d)
"than every / at least one"	`> ALL` / `> ANY`	P5, P6
"in each department, top k"	DENSE_RANK per department	P4, P10

✓ Practice MCQs (asked ones first)

1. Minimal super key is the: ✎ 49th BCS ICT 2025

(a) primary key (b) secondary key
(c) foreign key (d) candidate key

2. Cascadeless schedules prevent: ✎ 49th BCS ICT 2025

(a) deadlocks (b) dirty (uncommitted) reads
(c) aborts (d) long transactions

3. Bitmap indexes suit: ✎ 49th BCS ICT 2025

(a) low-cardinality columns (b) high-cardinality columns
(c) large text (d) frequent updates

4. Not an SQL keyword: ✎ 49th BCS ICT 2025

(a) ORDER BY (b) SELECT
(c) REMOVE (d) WHERE

5. Locking protocols are used for: ✎ BPDB AE(CSE) 2021

(a) recovery (b) indexing
(c) normalization (d) concurrency control

6. t(val) = 1, 2, 3, NULL, NULL, 4, 5. COUNT(val) = ✎ BCC AP 2021 (variant)

(a) 7 (b) 5
(c) 2 (d) 6

7. Degree of a relation = number of:

(a) rows (b) keys
(c) columns (d) tables

8. Removes all rows but keeps the table:

(a) TRUNCATE (b) DROP
(c) DELETE WHERE (d) ALTER

9. An M:N relationship becomes:

(a) one FK (b) a merged table
(c) an index (d) new table, composite PK

10. 3NF removes:

(a) repeating groups (b) transitive dep.
(c) partial dep. (d) all FDs

Answer key → 1-d · 2-b · 3-a · 4-c · 5-d · 6-b · 7-c · 8-a · 9-d · 10-b

Written question: Analyze the SQL on sheet 2 (Q1): what is wrong as printed, what does the corrected query return, and what changes with the own-department average? 10 marks

✎ **Asked in:** Combined Bank Officer(IT) 2023 · Rupali Bank ANE 2023

1. Errors: alias `e` undeclared → `FROM employees e`; `COUNT()` empty → `COUNT(*)` 2
2. WHERE keeps above-average employees (**before** grouping); GROUP BY department; HAVING keeps groups of more than 2 4
3. Output: department name and the average of those above-average salaries, highest first (sample: IT 70000) 2
4. Correlated form (`WHERE department_id = d.department_id`) uses the **own** department average → sample result empty 2

Before the exam ✓

- ✓ GROUP BY / HAVING from any schema
- ✓ self-join + Nth highest salary
- ✓ M:N ERD → tables + keys
- ✓ ACID with a bank transfer